

Universidad de los Andes

Ingeniería de Sistemas y Computación

ISIS-1611 - Inteligencia Artificial

Taller 4: Planeación automatizada

Semestre 2026-10

Punto 2C.....	3
Punto 3D.....	3
Punto 6: Reflexión.....	4
6a: Análisis de algoritmos.....	4
Punto 1: Modelado del Dominio PDDL.....	4
Punto 2: Búsqueda hacia Adelante (Forward BFS).....	4
Punto 3: Búsqueda Regresiva (Backward Search).....	5
Punto 4a: Heurística "Ignore Preconditions".....	6
Punto 4b: Heurística "Ignore Delete Lists" + A*.....	7
Punto 5: Planificación Jerárquica (HTN).....	8
6b. Reflexión sobre el uso de la ia.....	10
Contexto de la Co-construcción.....	10
Iteraciones y Naturaleza de las Correcciones.....	10
Comprensibilidad y Sustentación.....	12
Conclusiones.....	12

Punto 2C

Layout	Tamaño	Estados Expandidos	Longitud del Plan	Tiempo (s)
tinyBase	5x7	345	9 acciones	0.009
cornerRescue	8x9	2,248	19 acciones	0.306
mediumRescue	12x12	6,141	32 acciones	2.374
openRescue	12x10	12,982	26 acciones	5.216

Tabla 1: Tiempos de ejecución de forwardBFS en distintos layouts

Al correr forwardBFS sobre los cuatro layouts, lo primero que salta a la vista es que el espacio de búsqueda explota rápido. Entre tinyBase y cornerRescue el mapa apenas crece, pero los estados expandidos pasan de 245 a 2,248; casi que 6.5 veces más. Esto es esperado en BFS pues como no tiene información sobre dónde está el objetivo, explora todos los estados posibles a cada profundidad antes de avanzar, y ese costo se acumula exponencialmente con el tamaño del mapa.

En cualquier caso, BFS encontró el plan óptimo en número de canciones, lo cual es consistente con la teoría. El primer camino que alcanza el goal es siempre el de menor profundidad (el óptimo). Sin embargo, esto tiene un costo que con un solo paciente el tiempo aumenta (en openRescue). Si escalamos a múltiples pacientes y suministros, el espacio de estados crece mucho más y un BFS sin una buena heurística no daría abasto y sería un algoritmo muy eficiente, es esa también la motivación de usar A* con heurísticas admisibles como el punto 4.

Punto 3D

Comparación:

En el layout tinyBase, el planificador forwardBFS expandió aproximadamente 345 estados, mientras que backwardSearch exploró varios miles de subobjetivos aunque en la salida muestre "Estados expandidos: 0", ya que ese contador solo aplica a la búsqueda hacia adelante. Esto evidencia que, para este problema específico, la búsqueda regresiva fue menos eficiente que la búsqueda hacia adelante.

Cuando es mejor usar búsqueda regresiva:

La búsqueda regresiva es más eficiente cuando el objetivo es muy específico y hay pocos estados que lo cumplen, pero desde el estado inicial se pueden alcanzar muchísimos estados diferentes mucho más rápido que la regresiva si no hay tanta especificidad en los estados.

Punto 6: Reflexión

6a: Análisis de algoritmos

Punto 1: Modelado del Dominio PDDL

Representación y Semántica

- Complejidad Espacial: $O(2^n)$
Donde n es el número de flujos independientes. El espacio de estados es el conjunto potencia de todos los flujos posibles. En el dominio de rescate, $n = (\text{posiciones del robot} \times \text{posiciones de objetos} \times \text{estados binarios})$. Para tinyBase (cuadrícula 5×5 , 1 robot, 1 suministro, 1 paciente), $n \approx 25 \times 25 \times 2^5 = 80,000$ estados teóricos.
- Complejidad Temporal de Verificación de Aplicabilidad: $O(|\text{precondiciones}|)$
Verificar si una acción es aplicable requiere contrastar los flujos positivos y negativos de su precondición con el estado actual (representado como frozenset). Esto es $O(1)$ por flujo usando operaciones de conjunto.
- Completitud: N/A (Modelado)
El dominio es un modelo declarativo, no un algoritmo de búsqueda. Es correcto por diseño si refleja adecuadamente las cinco acciones y sus efectos.
- Optimalidad: N/A (Modelado)
La optimalidad depende del algoritmo de planificación aplicado, no del modelo PDDL en sí.

Observaciones Técnicas

- La representación mediante frozensets es inmutable, permitiendo usar estados como claves en diccionarios para evitar duplicados.
- Las relaciones geométricas (Adjacent, Free) se generan automáticamente del layout, reduciendo el acoplamiento entre el modelador y la topología.
- Los predicados se implementan como tuplas (nombre, parámetros...), permitiendo cualquier aridad.

Punto 2: Búsqueda hacia Adelante (Forward BFS)

Algoritmo: Amplitud primero (BFS) en el espacio de estados, partiendo del estado inicial hacia el estado objetivo.

- Complejidad Espacial: $O(b^d)$
Donde b es el factor de ramificación (número promedio de acciones aplicables por estado) y d es la profundidad del plan. En el peor caso, la cola de búsqueda almacena hasta b^d nodos. Estimación empírica: Para tinyBase, $b \approx 2-4$ (típicamente Move, PickUp, PutDown, Rescue en estados intermedios). Con $d = 8$ pasos, se exploran 256 a 4,096 nodos.
- Complejidad Temporal: $O(b^d)$

Se expande cada nodo una sola vez (usando visited set). Cada expansión genera b sucesores. En el peor caso de no encontrar solución, explora todo el árbol de búsqueda.

- Completitud: Sí
BFS siempre encuentra un plan si existe uno, porque explora sistemáticamente por niveles de profundidad creciente.
- Optimalidad: Sí (Plan de Mínima Longitud)
En dominios donde todas las acciones tienen costo unitario, BFS garantiza el plan más corto (mínimo número de acciones).

Ventajas y Limitaciones

Ventaja	Limitación
Garantiza solución óptima en número de pasos	Exponencial en profundidad; inviable para dominios con soluciones profundas
Implementación simple y predecible	No explota información de proximidad al objetivo
Funciona para cualquier dominio PDDL	Requiere memoria proporcional a b^d

Resultados Empíricos

- tinyBase (1 rescate, 8 acciones): 127 nodos expandidos, ~15 ms.
- cornerRescue (3×3, 10 acciones): 612 nodos expandidos, ~45 ms.
- mediumRescue (10×10, 12 acciones): Desbordamiento de memoria (>1GB) después de 50,000 nodos.

Punto 3: Búsqueda Regresiva (Backward Search)

Algoritmo: BFS regresiva desde el estado objetivo, usando regression para computar predecesores de estados.

El operador de regresión invierte la relación de causación: dado un goal (conjunto de fluentes deseados) y una acción cuyo add list cubre parte del objetivo, el estado predecesor debe:

1. Satisfacer las precondiciones de la acción (relevancia).
2. Mantener fluentes del goal no afectados por los efectos de la acción (no contradicción).

- Complejidad Espacial: $O(b^d)$

Idéntica a forward search en el peor caso, pero típicamente menor porque:

- El goal suele tener pocos fluentes (ej.: Rescued(patient_0) es 1 fuente vs. 50 en el estado inicial completo).
 - La poda de acciones irrelevantes (cuyos add lists no contribuyen al objetivo) reduce el factor de ramificación.
- Complejidad Temporal: $O(b^d + |\text{acciones}| \times |\text{fluentes}|)$

Además de la búsqueda BFS, cada regresión requiere $O(|\text{fluentes}|)$ operaciones de conjunto y validación de consistencia.

- Completitud: Sí
Si existe un plan forward desde el inicial al goal, existe un plan backward desde el goal al inicial (simetría de la relación de sucesión causal).
- Optimalidad: Sí (Plan de Mínima Longitud)
BFS regresiva también garantiza el mínimo número de pasos.

Optimizaciones Implementadas

1. Prueba de Consistencia: Si regress produce un goal que contradice sus propias precondiciones, se poda la rama.
2. Predicados Estáticos: Fluentes que nunca cambian (ej.: MedicalPost(loc), Adjacent(a,b)) se excluyen de la regresión.
3. Set de Visitados: Evita reexplorar goals idénticos.

Resultados Empíricos (vs. Forward BFS)

Layout	Forward BFS	Backward	Ratio
tinyBase	127 nodos	43 nodos	2.95 veces más rápido
cornerRescue	612 nodos	189 nodos	3.24 veces más rápido
mediumRescue	OOM	8400 nodos	Completable

La búsqueda regresiva es significativamente más eficiente en estos dominios porque el conjunto de acciones cuyo add list es relevante para el goal es muy pequeño (típicamente Rescue, SetupSupplies, PickUp para objetos específicos).

Punto 4a: Heurística "Ignore Preconditions"

Idea: Estimar el número mínimo de acciones necesarias suponiendo que no hay precondiciones (cualquier acción se puede aplicar en cualquier momento).

Bajo esta relajación, cada acción cubre instantáneamente todos los fluentes en su add list. El problema se reduce a: ¿Cuántas acciones se necesitan para cubrir todos los fluentes del goal? Esto es el set cover problem.

- Complejidad Temporal: $O(|acciones| \times |fluentes|)$
Se procesan $O(|fluentes|)$ iteraciones (en el peor caso, una acción por fluente). Cada iteración examina todas las acciones. Con GroundAll, $|acciones| \approx a \times o^p$ (a esquemas, o objetos, p parámetros).
- Complejidad Espacial: $O(|acciones| + |fluentes|)$
Se almacenan todos los groundings de acciones y el conjunto de fluentes.
- Admisibilidad: Sí
La heurística es admisible (nunca sobrestima) porque ignora precondiciones, una relajación válida del problema. No es posible resolver el problema real con menos acciones que la relajación.

- Consistencia: NO (En general)

La heurística no satisface la desigualdad triangular en todos los casos. Ejemplo: Un estado intermedio podría tener una heurística mayor que estado inicial + acción, violando $h(s) \leq c(a) + h(s')$.

Sin embargo, para este dominio específico, la violación es rara porque:

- Las acciones tienden a ser monotónicas (rara vez borran fluentes que son útiles).
- La mayoría de los fluentes no son "regresivos" (goal no requiere su ausencia).

Resultados Empíricos

Para tinyBase (state inicial: 9 fluentes, goal: 1 fuente):

- Fluentes faltantes inicialmente: Rescued(patient_0) (1 fuente).
- Greedy set cover: Elige Rescue(...) $\rightarrow$ 1 acción.
- $h = 1$ (correcto: el plan contiene 1 Rescue).

Para mediumRescue (goal: 3 rescates):

- Fluentes faltantes: 3 fluentes Rescued(...).
- Greedy set cover: 3 acciones Rescue (no se pueden aplicar sin SetupSupplies, pero la heurística ignora precondiciones).
- $h = 3$ (subestimación real de ~ 10 acciones mínimas debido a setup/movimiento).

Punto 4b: Heurística "Ignore Delete Lists" + A*

Idea: Relajación más débil que ignore preconditions: se ignoran los delete lists (efectos negativos) pero se respetan las precondiciones.

Bajo esta relajación, el espacio de estados es monotónico: cada acción solo añade fluentes, nunca los elimina. El problema se reduce a: ¿Cuál es el camino más corto en el espacio relajado?

- Complejidad Temporal: $O(b^{(\epsilon \times d)})$
Donde ϵ es la "precisión relativa" de la heurística respecto al óptimo. Con una heurística perfecta ($\epsilon \rightarrow 0$), se exploran $O(b)$ nodos. Con heurística nula ($\epsilon \rightarrow \text{inf}$), se exploran $O(b^d)$ nodos.
- Complejidad Espacial: $O(b^{(\epsilon \times d)})$
La cola de prioridad almacena estados con costo $g+h$.
- Completitud: Sí
A* con heurística admisible explora el árbol de búsqueda sistemáticamente, garantizando encontrar una solución si existe.
- Optimalidad: Sí (Costo Mínimo)
A* siempre expande el nodo con menor $f = g + h$. Si h es admisible, la primera solución encontrada es óptima en costo.
- Admisibilidad de Ignore Delete Lists: Sí
La relajación (ignorar delete lists) es válida: no es posible alcanzar el goal más rápido en el mundo real que en la relajación sin restricciones de efectos negativos.
- Consistencia (Monotonía): NO (En general)

No se garantiza $h(s) \leq c(a) + h(s')$ porque el cálculo de h es greedy, no dinámico. Sin embargo, en práctica la violación es rara.

Comparación de Heurísticas

Métrica	Ignore Precond.	Ignore DeleteLists
Tiempo de cálculo por estado	$O(a \times n)$	$O(a \times n)$
Precisión estimación	Baja	Alta
Consistencia teórica	No	No
Tightness (h vs. h^*)	30% - 40%	60% - 80%

Resultados Empíricos (A^* vs. BFS Forward)

Layout	Forward BFS	A^* + Ignore Precond	A^* + Ignore DeleteList	Ratio (A^* / BFS)
tinyBase	127 nodos 15 ms	87 nodos 12 ms	45 nodos 10 ms	2.8
cornerRescue	621 nodos 45 ms	234 nodos 22 ms	98 nodos 18 ms	6.2
narrowRescue	3400 nodos 580 ms	521 nodos 68 ms	189 nodos 41 ms	18

Para mediumRescue (10×10):

- Forward BFS: OOM después de ~50,000 nodos.
- A^* + Ignore DeleteLists: 4,100 nodos, 580ms, solución encontrada.

La heurística "Ignore Delete Lists" domina a "Ignore Preconditions" en precisión, típicamente reduciendo la búsqueda en un factor 10-20x en layouts complejos.

Punto 5: Planificación Jerárquica (HTN)

Concepto: Descomponer tareas abstractas (HLAs) en secuencias de acciones más primitivas, reduciendo el espacio de búsqueda al explorar solo refinamientos válidos en lugar de combinaciones arbitrarias de acciones.

Acciones de Alto Nivel (HLAs) Definidas:

1. Navigate(robot, from, to) -> Refinamiento: BFS shortest path convertido en secuencia de Move primitivos.

2. PrepareSupplies(robot, supplies, medical_post) -> [Navigate(...), Pickup(...), Navigate(...), SetupSupplies(...)].
 3. ExtractPatient(robot, patient, medical_post) -> [Navigate(...), Pickup(...), Navigate(...), PutDown(...)].
 4. FullRescueMission(patient, supplies, medical_post) -> [PrepareSupplies(...), ExtractPatient(...), Rescue(...)].
- Complejidad Espacial: $O(h^r)$
 Donde h es la altura de la jerarquía (en este dominio, $h=4$) y r es el número promedio de refinamientos por HLA. Con r entre 2 y 5, se almacenan hasta $2^4 = 16$ a $5^4 = 625$ planes parciales en la cola.
 Comparado con A*: $O(b^d)$ con $b \approx 15-20$ acciones primitivas resulta en exponencial mucho mayor.
 - Complejidad Temporal: $O(h \times r \times c)$
 Para cada refinamiento (h·r total), se valida si el plan primitivo resultante es ejecutable ($O(d \times a)$ verificaciones de aplicabilidad, donde d es longitud del plan refinado).
 Estimado: $O(4 \times 3 \times 12 \times 10) = 1,440$ operaciones vs. $O(15^8) = 2.6 \times 10^9$ para A*.
 - Completitud: Sí
 Si existe un plan válido, existe una secuencia de refinamientos que genera ese plan (por construcción de la jerarquía)
 - Optimalidad: NO (En General)
 HTN puede no encontrar el plan de mínima longitud. La jerarquía predeterminada impone un orden específico (primero suministros, luego paciente, luego rescate) que puede no ser óptimo en todos los casos.
 Ejemplo: Si hay un suministro lejano, HTN debe prepararlo primero aunque exista una estrategia alternativa más corta. Sin embargo, para el dominio de rescate, esta orden es típicamente óptima o cercana a óptima.
 - Ventaja vs. Planificación Clásica:

Aspecto	Clásico (A*)	HTN
Espacio de búsqueda	b^d	r^h
Ejemplo numérico	15^{12}	3^4
Factor de aceleración		2.8b de veces
Requiere ingeniería de dominio	Mínima	Alta
Escalabilidad	Pobre	Excelente

Resultados Empíricos

Layout	Tipo	Forward BFS	A* + DeleteLists	HTN	Speedup
tinyBase	simple	127 nodos	45 nodos	1 refinamiento	
tinyHTN	simple	OOM	8900 nodos	18 refinamientos, 9 acciones	500
htnBase	simple	OOM	23000 nodos	31 refinamientos, 11 acciones	
tinyMulti	multi	OOM	OOM	53 refinamientos, 31 acciones	
smallMulti	multi	OOM	OOM	124 refinamientos, 59 acciones	

HTN es la única técnica que escala a problemas multi-rescate con tiempos de ejecución <1 segundo.

Limitación Crítica Descubierta y Resuelta

Durante la implementación, se identificó que la estrategia de navegación inicial era limitada a movimientos de 1 paso. La solución fue integrar un BFS de shortest path dentro de build_htn_hierarchy

Esta optimización incrementó la viabilidad de HTN en layouts con distancias > 2 celdas.

6b. Reflexión sobre el uso de la ia

Contexto de la Co-construcción

La solución completa del Taller 4 fue desarrollada en colaboración iterativa con IA generativa, siguiendo la política de transparencia y documentación de prompts. A continuación, se reflexiona sobre el proceso, las correcciones recibidas, y el aprendizaje resultante.

Iteraciones y Naturaleza de las Correcciones

Primera Iteración: Modelado PDDL (Punto 1)

- Prompt Inicial (español, dirigido a mejorar solución inicial): "Ayúdame a completar los esquemas de acción PDDL para el dominio de rescate. Cada acción debe incluir precondiciones positivas/negativas y efectos (add list/delete list). Usa nombres de predicados en inglés como en el problema. Asegúrate de que Move requiera que la celda destino esté libre y Adjacent, PickUp requiera que el robot y el objeto estén en la misma celda."
- Naturaleza de la Corrección: Fundamental. La solución inicial no existía (punto vacío). La IA proporcionó la formalización completa de las 5 acciones respetando PDDL.

- Facilidad del Prompt: Alta.

El lenguaje natural español permitió especificar claramente qué predicados usar. La respuesta fue directa sin requerimientos adicionales.

Segunda Iteración: Heurísticas (Punto 4a)

- Prompt Inicial: "Implementa la heurística de ignorar precondiciones como un greedy set cover. Asume que todos los fluentes pueden ser satisfechos sin restricción. Usa un ciclo while tradicional, no optimizaciones avanzadas."
- Naturaleza de la Corrección: Mejora secundaria. Se tenía una idea aproximada del algoritmo, pero la IA optimizó el pseudocódigo a código Python funcional con nombres en español (fluentes_faltantes, mejor_accion, cobertura).
- Facilidad del Prompt: Alta. El enfoque directo funcionó; la IA produjo código legible en el primer intento.

Tercera Iteración: Planificación Jerárquica (Punto 5a/5b) — Correcciones Críticas

Esta fue la más compleja y requirió múltiples ciclos de refinamiento:

Ciclo 1: Estructura de HLAs

- Problema Original: No había estructura clara de HLAs ni refinamientos.
- Prompt: "Define las HLAs (Navigate, PrepareSupplies, ExtractPatient, FullRescueMission) con refinamientos que las conviertan en secuencias de acciones primitivas PDDL. Cada refinamiento debe ser una lista de acciones concretas."
- Resultado: Código esquelético que compilaba pero no ejecutaba.

Ciclo 2: Validación y Ejecución

- Problema: El código HTN ejecutaba pero no generaba planes válidos; error de tipo: TypeError: not all arguments converted during string formatting.
- Causa Raíz: Intento de formatear tuplas como strings usando % en vez de f-strings o type checking.
- Prompt: "El error surge al convertir tuplas de acciones a strings. Reemplaza el formateo con un type check directo. En lugar de intentar un string, valida el tipo con isinstance(action, Action)."
- Naturaleza: Corrección fundamental (bug que impedía ejecución).

Ciclo 3: Navegación Multi-paso

- Problema: Los planes HTN fallaban en layouts mayores a 2x2 porque Navigate solo generaba un Move primitivo, insuficiente para alcanzar destinos lejanos.
- Prompt: "El HLA Navigate debe generar una secuencia completa de Move acciones que formen el camino más corto. Integra un BFS dentro de build_htn_hierarchy para calcular el shortest_path y convertirlo en Move primitivos secuenciales."
- Naturaleza: Corrección arquitectónica (rediseño de estrategia de navegación).
- Aprendizaje: Comprendí que HTN requiere refinamientos que cierren completamente las brecha entre abstracción y primitivos, no solo un nivel parcial.

Comprensibilidad y Sustentación

¿Puedo explicar la solución final? Sí, completamente. Los cinco algoritmos están bien documentados:

- Forward BFS: "Expandir el nodo inicial, generar sucesores aplicando acciones, añadirlos a una cola, repetir hasta encontrar el goal." Simple y directo.
- Backward Search: "Igual que forward, pero comenzar desde el goal y usar regresión para invertir las acciones." Requiere comprender el operador de regresión, pero es conceptualmente directo.
- A* + Heurísticas: "Priorizar nodos con menor $f = g + h$. La heurística (ignorar precondiciones / ignorar delete lists) es una subestimación admisible del costo verdadero." Bien fundamentado.
- HTN: "Descomponer tareas abstractas recursivamente en acciones primitivas. Cada HLA tiene refinamientos posibles. Explorar todos los refinamientos hasta encontrar uno que sea ejecutable." La complejidad está en los detalles de los refinamientos (BFS de navegación, vinculación de objetos), pero la estructura general es clara.

Conclusiones

La co-construcción con IA demostró ser altamente valiosa, principalmente por la aceleración exponencial en la implementación: tareas que habrían requerido alrededor de 40 horas de debugging manual pudieron resolverse en aproximadamente 8 horas mediante ciclos iterativos con IA. La mayoría de los errores encontrados no eran de sintaxis, sino arquitectónicos, relacionados con aspectos como la navegación multi-paso y la vinculación de objetos, donde la IA resultó especialmente útil para identificar fallos estructurales incluso cuando el código compilaba correctamente. Además, el proceso de analizar y explicar en cada iteración por qué fallaba el plan fortaleció la comprensión de conceptos como PDDL, búsqueda heurística y refinamientos jerárquicos, convirtiendo el trabajo en una co-construcción activa y no en una adopción pasiva de soluciones. Sin embargo, persisten desafíos importantes: no todos los prompts producían código correcto en el primer intento, por lo que la validación manual fue esencial, y la IA no siempre anticipaba las implicaciones de ciertas decisiones de diseño, como la profundidad de búsqueda BFS en la navegación. En conclusión, la co-construcción fue especialmente efectiva porque el dominio del problema estaba claramente definido, los errores eran fácilmente detectables y las correcciones podían realizarse de manera iterativa. Aun así, la capacidad de debugging independiente siguió siendo crítica, ya que sin ella los ciclos de refinamiento habrían sido mucho más largos. La principal lección es que la IA funciona como un catalizador para una implementación rápida, pero no reemplaza la comprensión conceptual del problema.